CINELUX SYSTEM

REST API Documentation

Based on Component Diagram Architecture

Revision History

Ver	Date	Author	Change Summary
1.0	Apr 2026	Team	Initial API documentation generated from component diagram
1.1	Apr 2026	Team	Added FR4 GET /movies/{id}, QR token on booking, FR16 preferences, FR12 PUT/DELETE movies, clarified UUID vs int, JSONB fields, cold-start preferences

1. User API

Base Path: `/api/users` | Service: User Service → PostgreSQL Repository → PostgreSQL

Handles authentication, registration, profile management, and user preferences (cold-start onboarding).

FR COVERAGE

FR Coverage: FR1 (Register), FR2 (Login), FR5 (Manage Profile), FR16 (User Preferences / cold-start onboarding). Preference updates are also triggered server-side after each confirmed booking by the Recommendation Service.

1.1 Register User

POST	/api/users/register	Create a new user account with optional cold-start genre preferences

REQUEST BODY

```
{
  "firstName":       "Mais",
  "lastName":        "Al-Ahmad",
  "email":           "mais@example.com",
  "password":        "SecurePass!2026",
  "preferredGenres": ["Action", "Drama", "Sci-Fi"]
  // Optional — used by the Recommendation Service for cold-start onboarding (FR16)
}
```

PARAMETERS

Name	Type	Required	Description
`firstName`	string	**Yes**	User first name
`lastName`	string	**Yes**	User last name
`email`	string	**Yes**	Unique email address — triggers AUTH-002 if duplicate
`password`	string	**Yes**	Minimum 8 characters
`preferredGenres`	string[]	No	Genre list for cold-start recommendations; empty array is valid

RESPONSE 201 CREATED

```
{
  "userId":    "usr_9f4a2b1c",
  "firstName": "Mais",
  "lastName":  "Al-Ahmad",
  "email":     "mais@example.com",
  "createdAt": "2026-04-14T10:00:00Z"
}
```

NOTE

preferredGenres are stored in the user_preferences table and immediately seed the Recommendation Engine. If omitted, the system applies popularity-based defaults until booking history accumulates.

1.2 Login

POST	/api/users/login	Authenticate and receive a JWT bearer token

REQUEST BODY

```
{
  "email":    "mais@example.com",
  "password": "SecurePass!2026"
}
```

RESPONSE 200 OK

```
{
  "token":     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 3600,
  "userId":    "usr_9f4a2b1c"
}
```

1.3 Get User Profile

GET	/api/users/{userId}	Retrieve full profile for the authenticated user

PATH PARAMETERS

Name	Type	Required	Description
userId	string (UUID)	**Yes**	Target user identifier — must match the authenticated token subject

RESPONSE 200 OK

```
{
  "userId":          "usr_9f4a2b1c",
  "firstName":       "Mais",
  "lastName":        "Al-Ahmad",
  "email":           "mais@example.com",
  "preferredGenres": ["Action", "Drama"],
  "createdAt":       "2026-04-14T10:00:00Z"
}
```

1.4 Update User Profile

PUT	/api/users/{userId}	Update name, email, or password for the authenticated user

REQUEST BODY

```
{
  "firstName": "Mais",
  "lastName":  "Nasser",
  "email":     "mais.nasser@example.com",
  "password":  "NewPass!2026"
  // All fields optional — only provided fields are updated
}
```

RESPONSE 200 OK

```
{
  "userId":    "usr 9f4a2b1c",
  "firstName": "Mais",
  "lastName":  "Nasser",
  "email":     "mais.nasser@example.com",
  "updatedAt": "2026-04-14T11:00:00Z"
}
```

1.5 Update User Preferences

PUT	`/api/users/{userId}/preferences`	Explicitly update genre preferences for the recommendation engine (FR16)

✓ TRACEABILITY

FR16 — Preference updates occur in two ways: (1) explicitly via this endpoint, and (2) implicitly by the Recommendation Service after each confirmed booking. This endpoint is the explicit path; the implicit path requires no client action.

REQUEST BODY

```
{
  "preferredGenres": ["Thriller", "Comedy", "Animation"]
}
```

RESPONSE 200 OK

```
{
  "userId":          "usr_9f4a2b1c",
  "preferredGenres": ["Thriller", "Comedy", "Animation"],
  "updatedAt":       "2026-04-14T12:00:00Z"
}
```

2. Catalogues API

Base Path: `/api/movies` | Service: Catalogues Service → PostgreSQL Repository → PostgreSQL

Provides browsing, searching, and detail retrieval for movies, genres, and showtimes.

FR COVERAGE

FR Coverage: FR3 (Browse Catalogue), FR4 (View Movie Details — dedicated endpoint added per review), FR6 (Showtime Lookup). Movie fields (synopsis, posterUrl, rating, language, ageRating) are stored in the metadata JSONB column of the Movies table and are surfaced here for read operations.

2.1 Browse Movie Catalogue

GET	/api/movies	Return paginated list of movies with optional filters

QUERY PARAMETERS

Name	Type	Required	Description
genre	string	No	Filter by genre name (e.g. Action, Drama)
language	string	No	Filter by language (e.g. English, Arabic)
ageRating	string	No	Filter by age rating (e.g. PG, PG-13, R)
page	integer	No	Page number — default 1
limit	integer	No	Results per page — default 20, max 100

RESPONSE 200 OK

```
{
  "page":        1,
  "limit":       20,
  "totalCount": 87,
  "movies": [
    {
      "movieId":   "mov_abc123",
      "title":     "Dune: Part Three",
      "genre":     ["Sci-Fi", "Adventure"],
      "language":  "English",
      "ageRating": "PG-13",
      "rating":    8.4,
      "posterUrl": "https://cdn.cinema.io/posters/mov_abc123.jpg",
      "durationMinutes": 155
    }
  ]
}
```

NOTE

The genre, synopsis, posterUrl, rating, language, and ageRating fields are persisted inside the metadata JSONB column of the Movies table. The API layer extracts and promotes them to top-level response fields for client convenience.

2.2 Get Movie Details [FR4]

GET	/api/movies/{movieId}	Retrieve full detail for a single movie, including metadata used by the CBF recommendation engine

✓ TRACEABILITY

FR4 Fix — This dedicated endpoint was added following the review. GET /api/movies returns summary objects; this endpoint returns the complete record including awards, keywords, tags, and cast from the JSONB metadata column, which feed the Content-Based Filtering engine.

PATH PARAMETERS

Name	Type	Required	Description
movieId	string (UUID)	**Yes**	Unique movie identifier — triggers MOV-404 if not found

RESPONSE 200 OK

```
{
  "movieId":          "mov_abc123",
  "title":            "Dune: Part Three",
  "releaseDate":      "2026-12-18",
  "durationMinutes": 155,
  "genre":            ["Sci-Fi", "Adventure"],
  "language":         "English",
  "ageRating":        "PG-13",
  "rating":           8.4,
  "synopsis":         "Paul Atreides continues his journey across the desert planet Arrakis...",
  "posterUrl":        "https://cdn.cinema.io/posters/mov_abc123.jpg",
  "metadata": {
    "cast":     ["Timothée Chalamet", "Zendaya"],
    "keywords": ["desert", "empire", "prophecy"],
    "awards":   ["Cannes 2026 — Best Cinematography"],
    "tags":     ["epic", "space-opera", "dystopian"]
  }
}
```

2.3 Get Showtimes for a Movie

GET	/api/movies/{movieId}/showtimes	List all upcoming showtimes for the specified movie

PATH PARAMETERS

Name	Type	Required	Description
movieId	string (UUID)	**Yes**	Movie identifier — triggers MOV-404 if not found

QUERY PARAMETERS

Name	Type	Required	Description
date	string (ISO 8601)	No	Filter showtimes to a specific date — e.g. 2026-04-20
hall	string	No	Filter by hall/screen identifier

RESPONSE 200 OK

```
{
  "movieId": "mov_abc123",
  "showtimes": [
    {
      "showtimeId":  "show_001",
      "hallId":      "hall_03",
      "startTime":   "2026-04-20T19:30:00Z",
      "endTime":     "2026-04-20T22:05:00Z",
      "totalSeats":  120,
      "availableSeats": 43
    }
  ]
}
```

3. Booking API

Base Path: `/api/bookings` | Service: Booking Service → PostgreSQL Repository → PostgreSQL + Payment Gateway (simulated)

Manages seat reservation, ticket confirmation, QR code generation, booking retrieval, and cancellation.

FR COVERAGE

FR Coverage: FR7 (Create Booking), FR8 (Seat Selection), FR9 (View Booking), FR10 (QR Code — qrCodeToken included in booking response per review), FR11 (Cancel Booking — no refund logic). Payment Gateway integration is simulated: the service calls the gateway and receives a mock confirmation token.

3.1 Create Booking

POST	/api/bookings	Reserve selected seats, process simulated payment, and generate a QR-coded ticket

REQUEST BODY

```
{
  "userId":      "usr_9f4a2b1c",
  "showtimeId":  "show 001",
  "seatIds":     ["A12", "A13"]
}
```

PARAMETERS

Name	Type	Required	Description
`userId`	string (UUID)	**Yes**	Authenticated user — must match bearer token subject
`showtimeId`	string (UUID)	**Yes**	Target showtime — triggers SHOW-404 if not found
`seatIds`	string[]	**Yes**	One or more seat identifiers — triggers SEAT-409 if any seat is already taken

RESPONSE 201 CREATED

```
{
  "bookingId":      "bkg_7f3d2a",
  "userId":         "usr_9f4a2b1c",
  "showtimeId":     "show 001",
  "movieTitle":     "Dune: Part Three",
  "seats":          ["A12", "A13"],
  "status":         "CONFIRMED",
  "totalAmount":    24.00,
  "currency":       "USD",
  "paymentToken":   "PAY-SIM-99821",
  "qrCodeToken":    "QR-bkg_7f3d2a-2026041419300",
  "qrCodeUrl":      "https://cdn.cinema.io/qr/bkg_7f3d2a.png",
  "createdAt":      "2026-04-14T19:30:00Z"
}
```

✓ TRACEABILITY

FR10 Fix — qrCodeToken is a unique opaque string linking this booking to the Validation Service. qrCodeUrl is the rendered PNG served to the client. Both are generated at booking creation and stored in the bookings table.

3.2 Get Booking Details

GET	/api/bookings/{bookingId}	Retrieve full details of a specific booking

PATH PARAMETERS

Name	Type	Required	Description
bookingId	string (UUID)	Yes	Booking identifier — triggers BOOK-404 if not found

RESPONSE 200 OK

```
{
  "bookingId":    "bkg_7f3d2a",
  "userId":       "usr_9f4a2b1c",
  "movieTitle":   "Dune: Part Three",
  "showtimeId":   "show_001",
  "startTime":    "2026-04-20T19:30:00Z",
  "seats":        ["A12", "A13"],
  "status":       "CONFIRMED",
  "qrCodeToken":  "QR-bkg_7f3d2a-2026041419300",
  "qrCodeUrl":    "https://cdn.cinema.io/qr/bkg_7f3d2a.png",
  "createdAt":    "2026-04-14T19:30:00Z"
}
```

3.3 List User Bookings

GET	/api/bookings	Return all bookings for the authenticated user

QUERY PARAMETERS

Name	Type	Required	Description
userId	string (UUID)	Yes	Filter bookings by user — must match bearer token subject
status	string	No	Filter by status: CONFIRMED | CANCELLED | USED
page	integer	No	Page number — default 1
limit	integer	No	Results per page — default 20

RESPONSE 200 OK

```
{
  "page":  1,
  "total": 4,
  "bookings": [
    {
      "bookingId":  "bkg_7f3d2a",
      "movieTitle": "Dune: Part Three",
      "startTime":  "2026-04-20T19:30:00Z",
      "seats":      ["A12", "A13"],
      "status":     "CONFIRMED",
      "qrCodeUrl":  "https://cdn.cinema.io/qr/bkg_7f3d2a.png"
    }
```

```
  ]
}
```

3.4 Cancel Booking

DELETE	/api/bookings/{bookingId}	Cancel a booking, update status to CANCELLED, and release held seats

> ⚠ **NOTE**
> FR11 — Cancellation is intentionally simple per system specification: status is set to CANCELLED, seats are released in the seat map, and no refund or billing logic is executed. Payment Gateway is not called on cancellation.

PATH PARAMETERS

Name	Type	Required	Description
bookingId	string (UUID)	**Yes**	Booking to cancel — triggers BOOK-404 if not found, BOOK-403 if cancellation is not permitted

RESPONSE 200 OK

```
{
  "bookingId":   "bkg_7f3d2a",
  "status":      "CANCELLED",
  "cancelledAt": "2026-04-15T08:00:00Z",
  "seatsReleased": ["A12", "A13"]
}
```

4. Admin API

Base Path: `/api/admin` | Service: Admin Service → PostgreSQL Repository → PostgreSQL

Restricted to users with the ADMIN role. Manages the cinema catalogue: movies, showtimes, and hall configuration.

FR COVERAGE

FR Coverage: FR12 (Manage Movies — POST, PUT, DELETE added per review), FR13 (Manage Showtimes), FR14 (Manage User Accounts). All endpoints require Authorization: Bearer <admin-token>.

4.1 Add Movie

POST	/api/admin/movies	Create a new movie record in the catalogue

REQUEST BODY

```
{
  "title":            "Oppenheimer II",
  "releaseDate":      "2027-07-21",
  "durationMinutes": 140,
  "genre":            ["Drama", "History"],
  "language":         "English",
  "ageRating":        "PG-13",
  "synopsis":         "The aftermath of Trinity...",
  "posterUrl":        "https://cdn.cinema.io/posters/op2.jpg",
  "metadata": {
    "cast":     ["Cillian Murphy"],
    "keywords": ["bomb", "Cold War"],
    "tags":     ["biopic", "historic"]
  }
}
```

NOTE

genre, synopsis, posterUrl, language, ageRating, and metadata are stored inside the movies.metadata JSONB column. The API accepts them as structured fields and serialises them before persistence.

RESPONSE 201 CREATED

```
{
  "movieId":   "mov_def456",
  "title":     "Oppenheimer II",
  "createdAt": "2026-04-14T09:00:00Z"
}
```

4.2 Update Movie [FR12]

PUT	/api/admin/movies/{movieId}	Update an existing movie record — partial updates supported

✓ TRACEABILITY

FR12 Fix — This endpoint was missing in v1.0. It allows admin operators to correct movie information, update metadata, or change availability status without deleting and recreating the record.

PATH PARAMETERS

Name	Type	Required	Description
movieId	string (UUID)	**Yes**	Movie to update — triggers MOV-404 if not found

REQUEST BODY (ALL FIELDS OPTIONAL)

```
{
  "title":           "Oppenheimer II: The Return",
  "durationMinutes": 148,
  "ageRating":       "R",
  "synopsis":        "Updated synopsis...",
  "metadata": {
    "tags": ["biopic", "historic", "thriller"]
  }
}
```

RESPONSE 200 OK

```
{
  "movieId":   "mov_def456",
  "title":     "Oppenheimer II: The Return",
  "updatedAt": "2026-04-14T10:30:00Z"
}
```

4.3 Delete Movie [FR12]

DELETE	/api/admin/movies/{movieId}	Permanently remove a movie and all associated showtimes

✓ TRACEABILITY

FR12 Fix — Soft-delete is recommended for production to preserve booking history integrity. This endpoint performs a logical delete by setting an is_active flag. Related showtimes are deactivated; existing confirmed bookings are unaffected.

PATH PARAMETERS

Name	Type	Required	Description
movieId	string (UUID)	**Yes**	Movie to delete — triggers MOV-404 if not found

RESPONSE 200 OK

```
{
  "movieId":   "mov_def456",
  "deleted":   true,
  "deletedAt": "2026-04-14T11:00:00Z"
}
```

4.4 Add Showtime

POST	/api/admin/showtimes	Schedule a new showtime for an existing movie

REQUEST BODY

```
{
  "movieId":   "mov_abc123",
  "hallId":    "hall_03",
  "startTime": "2026-05-01T20:00:00Z",
  "totalSeats": 120
}
```

RESPONSE 201 CREATED

```
{
  "showtimeId": "show_009",
  "movieId":    "mov_abc123",
  "hallId":     "hall_03",
  "startTime":  "2026-05-01T20:00:00Z",
  "endTime":    "2026-05-01T22:35:00Z",
  "totalSeats": 120,
  "createdAt":  "2026-04-14T09:15:00Z"
}
```

4.5 Manage User Account Status

PATCH	/api/admin/users/{userId}	Activate, suspend, or modify the role of a user account

PATH PARAMETERS

Name	Type	Required	Description
userId	string (UUID)	**Yes**	Target user — triggers GEN-500 if not found

REQUEST BODY

```
{
  "status": "SUSPENDED",
  "role":   "ADMIN"
  // Both fields optional — provide only what changes
}
```

RESPONSE 200 OK

```
{
  "userId":    "usr_9f4a2b1c",
  "status":    "SUSPENDED",
  "role":      "ADMIN",
  "updatedAt": "2026-04-14T12:00:00Z"
}
```

5. Recommendation API

Base Path: `/api/recommendations` | Service: Recommendation Service → PostgreSQL Repository → PostgreSQL

Returns personalised movie recommendations using a Content-Based Filtering (CBF) engine seeded by user preferences and booking history.

FR COVERAGE

FR Coverage: FR15 (Personalised Recommendations — UC-01.1). The engine reads preferredGenres, booking history, and movie metadata (keywords, tags) from PostgreSQL. Cold-start users (no history) receive recommendations based on their preferredGenres registered at sign-up.

5.1 Get Personalised Recommendations

GET	`/api/recommendations/{userId}`	Return a ranked list of recommended movies for the specified user

PATH PARAMETERS

Name	Type	Required	Description
`userId`	string (UUID)	**Yes**	User whose preference profile is used — triggers GEN-500 if the engine fails

QUERY PARAMETERS

Name	Type	Required	Description
`limit`	integer	No	Number of recommendations to return — default 10, max 50
`genre`	string	No	Optionally constrain recommendations to a specific genre

RESPONSE 200 OK

```
{
  "userId": "usr_9f4a2b1c",
  "recommendations": [
    {
      "movieId":          "mov_abc123",
      "title":            "Dune: Part Three",
      "genre":            ["Sci-Fi", "Adventure"],
      "rating":           8.4,
      "posterUrl":        "https://cdn.cinema.io/posters/mov_abc123.jpg",
      "relevanceScore":   0.94,
      "reason":           "Based on your interest in Sci-Fi and past bookings"
    }
  ],
  "coldStart": false
}
```

NOTE

coldStart: true is returned when the user has no booking history and recommendations are derived solely from preferredGenres registered at sign-up. The client may display a prompt encouraging the user to update preferences.

6. Error Model

All endpoints return errors in the following unified format. Clients should parse errorCode for programmatic handling.

UNIFIED ERROR RESPONSE SCHEMA

```
{
  "errorCode": "string",
  "message":   "string",
  "details":   "string (optional)"
}
```

EXAMPLE — SEAT-409

```
{
  "errorCode": "SEAT-409",
  "message":   "Seat already booked.",
  "details":   "Seat A12 is unavailable for showtime show_001."
}
```

ERROR CODE REFERENCE

Error Code	HTTP Status	Description
AUTH-001	401	Invalid credentials — email/password mismatch
AUTH-002	409	Registration conflict — email address already registered
MOV-404	404	Movie not found — no record matching the provided movieId
SHOW-404	404	Showtime not found — no record matching the provided showtimeId
SEAT-409	409	Seat already booked — requested seat is not available for this showtime
BOOK-404	404	Booking not found — no record matching the provided bookingId
BOOK-403	403	Cancellation not allowed — booking is outside the cancellation window or already cancelled
REC-500	500	Recommendation engine error — AI/CBF engine failed to return results
GEN-500	500	Internal server error — unhandled exception in the service layer

7. Identifier Convention

⚠ NOTE

Review Note — ID type consistency: The ERD defines integer primary keys (movie_id INT, user_id INT). This API documentation uses string UUIDs (mov_abc123, usr_9f4a2b1c) for all identifiers. DECISION REQUIRED: Choose one of the following and apply it consistently across the ERD, API, and database schema: Option A — UUIDs (recommended for distributed systems, URL-safe, no enumeration risk). Update the ERD to use UUID types. Option B — Integer IDs (simpler for internal references, smaller storage). Update this API documentation to use integer path parameters. Until resolved, this document assumes UUIDs as the canonical identifier type.